OpenMP

Complete Study Guide

Shared Memory Parallel Programming

High Performance Computing Module | 4th Year AI Course

Dept. of Artificial Intelligence

PART I

Foundations of OpenMP

Parallel regions · Work sharing · Data environment · Basic synchronization

1. Why Parallel Programming Matters

1.1 The Hardware Reality

Contemporary servers typically contain 2 to 8 (or more) symmetric multiprocessors (SMPs), and modern processors are multicore by design. This means that a program running on a single thread is, by definition, leaving most of the machine idle. Achieving high performance today requires exploiting parallelism — it is no longer optional.

1.2 Shared Memory and Threads

The natural programming model for SMP machines is **multithreading**. In this model:

- All processors share the same physical memory.
- Multiple threads exist within a single process and therefore see the **same address space**.
- Threads can communicate simply by reading and writing shared variables.

This is fundamentally different from distributed-memory programming (e.g., MPI), where each process has its own private memory and must explicitly send messages to exchange data.

1.3 Why Multithreading Is Hard

Despite its conceptual simplicity, multithreading is widely considered difficult in practice for several reasons:

- **Race conditions:** Two threads read and write the same variable concurrently, producing non-deterministic, incorrect results.
- **Deadlocks:** Two threads each wait for a resource held by the other, causing the program to hang forever.
- **Non-determinism:** Bugs appear and disappear depending on scheduling order, making them extremely hard to reproduce and debug.
- **Boilerplate overhead:** Even parallelizing a simple loop requires significant effort. Different threading APIs: Windows: `CreateThread`, UNIX: `pthread_create`. Need mutex to protect shared accesses. Different code for serial and parallel versions.

Consider the following code to parallelize using threads:

Simple loop — a lot of work to do a simple thing with raw threads

```
for (i = 0; i < n; i++) {
    sum = sum + sqrt(sin(data[i]));
}
// Problems:
// - Need mutex to protect accesses to sum
// - Different code for serial and parallel version
// - Different API per OS (pthread_create vs CreateThread)
```

> ⚡ **Key Insight**
>
> OpenMP is a language extension that introduces parallelization constructs into the language. Parallelization is **orthogonal** to the functionality — if the compiler does not recognize the OpenMP directives, the code remains functional (albeit single-threaded).

2. What Is OpenMP?

OpenMP is a **language extension** — implemented via compiler directives, library functions, and environment variables — that adds parallelization constructs to C, C++, and Fortran. Key design principles:

- **Orthogonality to functionality:** OpenMP directives are written as `#pragma` statements, which standard-compliant compilers silently ignore. The same source file compiles and runs correctly as a serial program if OpenMP is not enabled.
- **Based on shared-memory multithreaded programming:** all processors share the same memory.
- **Rich construct set:** Includes constructs for parallel programming: critical sections, atomic access, variable privatization, barriers, etc.
- **Industry standard:** Supported by Intel, Microsoft, Sun, IBM, HP, etc. Available for C++ and FORTRAN. Some behavior is implementation-dependent.

Category	Purpose
Parallel Regions	Define code to run in parallel
Work Sharing	Distribute work among threads
Data Environment	Control variable visibility and scope
Synchronization	Coordinate thread execution
Runtime Functions / Env Vars	Configure OpenMP behavior at runtime (several `omp_<something>` function calls and `OMP_<something>` environment variables)

3. The OpenMP Execution Model: Fork-Join

Fork and Join: Master thread spawns a team of threads as needed.

- The program begins with a single **master thread**.
- When a parallel region is encountered, the master thread **forks** a team of worker threads.
- All threads in the team execute the parallel region concurrently.
- At the end of the parallel region, threads **join** back — they synchronize and terminate, leaving only the master thread to continue.
- This can happen multiple times throughout the program.

Fork-Join model (schematic)

```
Master Thread ──────────────────────────────────────────────────►
                    │                           │
         FORK ──────┤              FORK ────────┤
               Worker Worker            Worker Worker
               Thread Thread            Thread Thread
                 1      2                 1      2
         JOIN ──────┤              JOIN ────────┤
```

This model is simple and predictable: parallelism is explicit and scoped to clearly defined regions.

4. OpenMP Memory Model and Syntax

4.1 Memory Model

OpenMP uses a **shared memory model**:

- Threads communicate by accessing **shared variables**.
- The sharing is defined **syntactically** — any variable that is seen by two or more threads is shared; any variable seen by one thread only is private.
- **Race conditions are possible** — use synchronization to protect from conflicts, or change how data is stored to minimize the need for synchronization.

4.2 General Syntax

Most of the constructs of OpenMP are pragmas:

General pragma syntax

```
#pragma omp construct [clause [clause] ...]
```

An OpenMP construct applies to a **structural block** — a block of code with one entry point and one exit point (essentially a single statement or a `{ }` block).

5. Parallel Regions

The `#pragma omp parallel` directive causes every thread in the team to execute the enclosed block independently and simultaneously:

Basic parallel region — every thread executes this block

```
double D[1000];
#pragma omp parallel
{
    int i; double sum = 0;
    for (i = 0; i < 1000; i++) sum += D[i];
    printf("Thread %d computes %f\n",
           omp_get_thread_num(), sum);
}
```

Key observations:

- **D is shared:** declared outside the parallel block, so all threads see the same array in memory.
- **i and sum are private:** declared inside the `{ }` block, so each thread gets its own independent copy. This is the **syntactic** definition of privacy in OpenMP.
- `omp_get_thread_num()` returns the ID of the calling thread (0 through N-1).
- `omp_set_num_threads(n)` or the environment variable `OMP_NUM_THREADS` controls how many threads are spawned.
- Running the same computation redundantly on every thread is rarely the goal — use `omp_get_thread_num()` to distribute work between threads.

6. Work Sharing Constructs

Work sharing is how OpenMP divides labor among threads so each thread does a different piece of the total work.

6.1 Sections: Independent Tasks

When you have two or more independent computations that can run simultaneously, use `#pragma omp sections`:

Before: sequential — two independent computations

```
answer1 = long_computation_1();
answer2 = long_computation_2();
if (answer1 != answer2) { ... }
```

After: parallelized with omp sections

```
#pragma omp sections
{
    #pragma omp section
    answer1 = long_computation_1();

    #pragma omp section
    answer2 = long_computation_2();
}
if (answer1 != answer2) { ... }
```

Each `section` block is assigned to a different thread. The two functions run in parallel. After the `sections` block, all results are available. Ideal for **task-level parallelism** — a small number of distinct independent tasks.

6.2 Parallel For Loops

The most common pattern in scientific computing is a loop whose iterations are independent. OpenMP offers three levels of control:

Level 1 — Sequential (baseline)

```
for (int i = 0; i < N; i++) { a[i] = b[i] + c[i]; }
```

Level 2 — Semi-manual parallelization

```
#pragma omp parallel
{
    int id = omp_get_thread_num();
    int Nthr = omp_get_num_threads();
    int istart = id * N / Nthr;
    int iend = (id + 1) * N / Nthr;
    for (int i = istart; i < iend; i++) {
        a[i] = b[i] + c[i];
    }
}
```

Level 3 — Automatic parallelization (preferred)

```
#pragma omp parallel
#pragma omp for schedule(static)
{
    for (int i = 0; i < N; i++) { a[i] = b[i] + c[i]; }
}
// Combined shorthand:
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++) { a[i] = b[i] + c[i]; }
```

6.3 Notes on #pragma omp for — Supported Loop Forms

Only simple kinds of for loops are supported. Requirements:

- One **signed integer variable** in the loop.
- **Initialization:** `var = init`
- **Comparison:** `var op last`, where op is: `<`, `>`, `<=`, `>=`
- **Increment:** `var++`, `var--`, `var += incr`, `var -= incr`, etc.
- All of `init`, `last`, `incr` must be **loop invariant** (not modified inside the loop).

Can combine the parallel and work sharing directives: `#pragma omp parallel for ...`

6.4 Problems of #pragma omp parallel for

Two key problems arise:

- **Load balancing:** If all iterations execute at the same speed, processors are used optimally. If some iterations are faster than others, some processors may go idle, reducing speedup. We don't always know the distribution of work — may need to re-distribute dynamically.
- **Granularity:** Thread creation and synchronization takes time. Assigning work to threads on per-iteration resolution may take more time than the execution itself. Need to coalesce work into coarse chunks to overcome threading overhead.

There is always a **trade-off between load balancing and granularity**.

7. Controlling Work Distribution: The schedule Clause

7.1 schedule(static [, chunksize])

Default: chunks of approximately equivalent size, one per thread. If more chunks than threads, assigned in round-robin to threads.

- **Advantage:** Zero runtime overhead for scheduling.
- **Best for:** Loops where all iterations take roughly the same time.

7.2 schedule(dynamic [, chunksize])

Threads receive chunk assignments **dynamically** at runtime as they become free. Default chunk size = 1.

- **Advantage:** Naturally handles load imbalance — fast threads pick up more chunks.
- **Cost:** Higher overhead due to runtime coordination.
- **Best for:** Loops with irregular or unpredictable iteration costs.

7.3 schedule(guided [, chunksize])

Threads receive chunks dynamically. Chunk size **reduces exponentially** (starts large, shrinks down to `chunksize`). A hybrid strategy reducing overhead while improving balance near the end.

- **Best for:** Loops where load imbalance is moderate.

> ⚡ **Key Insight**
>
> There is always a trade-off between load balancing (favoring dynamic, small chunks) and granularity (favoring static, large chunks). Choose your schedule based on the nature of the computation.

7.4 Controlling Granularity

Disable parallelism below a threshold / override thread count

```
#pragma omp parallel if (expression)
// Can be used to disable parallelization when the input is
// determined to be too small to be beneficially multithreaded.

#pragma omp num_threads (expression)
// Control the number of threads used for this parallel region.
```

8. Data Environment

OpenMP uses a **shared memory programming model**. Most variables (including locals) are shared by default:

- Global variables are always shared.
- Variables declared outside the parallel block are shared.
- Automatic variables inside the statement block are private.
- Automatic variables in called functions are private.
- Variables can be **explicitly declared as private** — a local copy is created for each thread.

Dangerous example — race condition on sum

```
{
    int sum = 0;
    #pragma omp parallel for
    for (int i = 0; i < N; i++) sum += i;
    // RACE CONDITION! Multiple threads read/modify/write sum
}
```

8.1 Overriding Storage Attributes

Clause	Behaviour
private(var)	A copy of the variable is created for each thread. There is no connection between the original variable and the private copies. Can achieve the same using variables inside { }.
firstprivate(var)	Same as private, but the initial value of the variable is copied from the main copy at the start of the parallel region.
lastprivate(var)	Same as private, but the last value of the variable (from the last sequential iteration) is copied back to the main copy after the parallel region ends.

Storage attribute clause examples

```
// private — loop variable is private by default
int i;
#pragma omp parallel for private(i)
for (i = 0; i < n; i++) { ... }

// firstprivate + lastprivate
int idx = 1;
int x = 10;
#pragma omp parallel for \
    firstprivate(x) lastprivate(idx)
for (i = 0; i < n; i++) {
    if (data[i] == x) idx = i;
}
// After loop: idx holds result from the last sequential iteration
```

8.2 Reduction

A very common pattern in scientific code is accumulating a result across all iterations:

Problem — sum is not private, but atomic access is too expensive

```
for (j = 0; j < N; j++) {
    sum = sum + a[j] * b[j];
```

```
}
```

Solution: have a private copy of sum in each thread, then add them up. Use the **reduction clause**:

Correct and efficient parallel reduction using reduction clause

```
#pragma omp parallel for reduction(+: sum)
for (j = 0; j < N; j++) {
    sum = sum + a[j] * b[j];
}
// Any associative operator can be used: +, -, ||, |, *, &&, ^, etc.
// The private value is initialized automatically (0 for +, 1 for *, ~0 for |,
...)
```

OpenMP internally:

- Creates a **private copy** of sum per thread, initialized to the identity value (0 for +, 1 for *).
- Each thread accumulates into its own private copy independently — **no synchronization during the loop**.
- At the end, all private copies are **combined** using the operator into the original sum.

9. OpenMP Synchronization

Race condition example — what should X be with 2 threads?

```
X = 0;
#pragma omp parallel
X = X + 1;
// Expected: 2. But can be 1 with unfortunate interleaving.
// OpenMP assumes the programmer knows what (s)he is doing.
```

OpenMP assumes that the programmer knows what they are doing. Regions marked to run in parallel are assumed independent. If access collisions are possible, it is the **programmer's responsibility** to insert protection.

9.1 Available Synchronization Mechanisms

- Single/Master execution
- Critical sections, Atomic updates
- Ordered
- Barriers
- Nowait (turn synchronization off)
- Flush (memory subsystem synchronization)
- Reduction (already covered)

9.2 Single and Master

Feature	#pragma omp single	#pragma omp master
Executor	First available thread	Thread 0 (Master) only
Barrier	Implicit barrier at end	No barrier
Use Case	Thread-safe I/O, Init	Logging, Master-worker logic

Single and Master example

```
#pragma omp parallel
{
    do_preprocessing();
    #pragma omp single
    read_input();            // Only one thread reads; rest wait at implicit
barrier
    #pragma omp master
    notify_input_consumed(); // Only master notifies; no barrier involved
    do_processing();
}
```

9.3 Critical Sections

`#pragma omp critical [name]` — standard critical section functionality.

- Critical sections are **global** in the program — can be used to protect a single resource in different functions.
- Critical sections are identified by **name** — all unnamed critical sections are mutually exclusive throughout the entire program.
- All critical sections with the **same name** are mutually exclusive between themselves.
- Named: `critical(name)` allows distinct sections to run concurrently. **Avoid large critical sections** — the loop becomes effectively serial.

9.4 Atomic Execution

Critical sections on the cheap. Protects a **single variable update**. Can be much more efficient (a dedicated assembly instruction on some architectures).

Atomic syntax and constraints

```
#pragma omp atomic
update_statement
// Update statement is one of:
//   var = var op expr, var op= expr, var++, var--
// The variable must be a scalar.
// The operation op is one of: +, -, *, /, ^, &, |, <<, >>
// The evaluation of expr is NOT atomic!
```

9.5 Ordered Execution

Ordered — executes statement in sequential order of iterations

```
#pragma omp parallel for
for (j = 0; j < N; j++) {
    int result = heavy_computation(j);  // Runs in parallel
    #pragma omp ordered
    printf("computation(%d) = %d\n", j, result); // Output in order
}
```

9.6 Barrier Synchronization

`#pragma omp barrier` — performs a barrier synchronization between all threads in a team at the given point.

Barrier example — ensure all threads have updated sum before using it

```
#pragma omp parallel
{
    int result = heavy_computation_part1();
    #pragma omp atomic
    sum += result;
    #pragma omp barrier           // Wait until ALL threads have updated sum
    heavy_computation_part2(sum); // Now safe to use the complete sum
}
```

10. OpenMP Runtime System and Functions

10.1 Thread Lifecycle

Each `#pragma omp parallel` creates a team of threads, which exist as long as the following block executes.

- `#pragma omp for` and `#pragma omp section` must be placed **dynamically within** a `#pragma omp parallel`.
- **Optimization:** If there are several `#pragma omp for` and/or `#pragma omp section` within the same parallel region, threads are **not destroyed and created again** — they are reused.
- A `#pragma omp for` is **not permitted** within the dynamic extent of another `#pragma omp for` — must include the inner `#pragma omp for` within its own `#pragma omp parallel`.
- **Nested parallelism** — the effect is implementation-dependent (may or may not create a new set of threads).

10.2 Runtime Control Functions

Function	Description
omp_set_dynamic(int) / omp_get_dynamic()	Allows the implementation to adjust the number of threads dynamically.
omp_set_num_threads(int) / omp_get_num_threads()	Control the number of threads for parallelization (maximum in case of dynamic adjustment). Must be called from sequential code. Also set by OMP_NUM_THREADS environment variable.
omp_get_num_procs()	How many processors are currently available?
omp_get_thread_num()	Returns the calling thread's ID (0 through N-1).
omp_set_nested(int) / omp_get_nested()	Enable/query nested parallelism.
omp_in_parallel()	Returns nonzero if currently running in parallel mode.
omp_get_wtime()	A portable way to compute wall clock time for benchmarking.

10.3 Explicit Locking

Can be used to pass lock variables around (unlike critical sections). Can implement more involved synchronization constructs. Use `#pragma omp flush` to synchronize memory.

Explicit lock functions

```
omp_lock_t my_lock;
omp_init_lock(&my_lock);     // Initialize
omp_set_lock(&my_lock);      // Acquire (blocking)
// ... protected work ...
omp_unset_lock(&my_lock);    // Release
omp_destroy_lock(&my_lock); // Cleanup

// omp_test_lock() — non-blocking try-acquire
```

11. Worked Example: Computing Pi via Numerical Integration

11.1 The Mathematics

The value of pi can be approximated using the following integral:

pi = integral from 0 to 1 of 4.0 / (1 + x^2) dx

This is approximated numerically by the **rectangle rule**: divide [0,1] into `num_steps` rectangles:

- Width: `step = 1.0 / num_steps`
- Midpoint of rectangle i: `x = (i + 0.5) * step`
- Height: `f(x) = 4.0 / (1.0 + x*x)`
- pi ≈ step × sum of all heights

11.2 Serial Implementation

Sequential Pi computation

```
static long num_steps = 100000;
double step, pi;

void main()
{
    int i;
    double x, sum = 0.0;
    step = 1.0 / (double) num_steps;

    for (i = 0; i < num_steps; i++) {
        x = (i + 0.5) * step;
        sum = sum + 4.0 / (1.0 + x * x);
    }
    pi = step * sum;
    printf("Pi = %f\n", pi);
}
```

11.3 Variable Analysis — Before Parallelizing

Ask: What variables can be shared? What variables need to be private? What variables should be set up for reductions?

Variable	Role	Sharing Strategy
num_steps, step	Loop-invariant constants	shared (read-only, safe for all threads)
pi	Final result, written once after loop	shared (written after loop ends)
i	Loop index	private (automatically — it is the loop variable)
x	Per-iteration midpoint (i+0.5)*step	private (different value each iteration)
sum	Accumulated total across all iterations	reduction(+:sum) — private copies merged at end

11.4 Parallel Implementation

Parallel Pi computation — minimal change from serial version

```
static long num_steps = 100000;
double step, pi;

void main()
{
    int i;
    double x, sum = 0.0;
    step = 1.0 / (double) num_steps;

    #pragma omp parallel for \
        private(x) reduction(+:sum)
    for (i = 0; i < num_steps; i++) {
        x = (i + 0.5) * step;
        sum = sum + 4.0 / (1.0 + x * x);
    }
    // i is private since it is the loop variable

    pi = step * sum;
    printf("Pi = %f\n", pi);
}
```

Pragma breakdown:

- **`#pragma omp parallel for`** — creates a thread team and distributes loop iterations automatically.
- **`private(x)`** — each thread gets its own copy of x, avoiding a race condition on the midpoint calculation.
- **`reduction(+:sum)`** — each thread maintains a partial sum; all partial sums are added together into the final sum at the end. Loop variable i is implicitly private.

> ⚡ **Key Insight**
>
> The result is a clean, correct parallel program that is almost identical in structure to the serial version. This is the power of OpenMP — minimal code change for significant parallelism.

12. Nested Parallelism and Runtime Considerations

Each `#pragma omp parallel` creates a thread team for the duration of the enclosed block. If multiple `#pragma omp for` loops appear within the same parallel region, threads are **reused** rather than destroyed and recreated — an important optimization.

A `#pragma omp for` **cannot** be nested directly inside another `#pragma omp for` in the same parallel region. To parallelize nested loops, wrap the inner loop in its own `#pragma omp parallel`. Nested parallelism can be enabled via `omp_set_nested(1)`, but its effect is implementation-dependent (may or may not create a new set of threads).

`#pragma omp flush [(var-list)]` forces a thread to synchronize its memory view with the shared store — equivalent to a memory fence. Most synchronization constructs (barriers, critical sections) include an **implicit flush**.

PART II

Advanced OpenMP Concepts

Synchronization internals · False sharing · SIMD vectorization · Task-based parallelism

13. Synchronization: Necessity and Cost

13.1 The Problem: Race Conditions

Occur when multiple threads access shared memory concurrently and at least one access is a write:

- **Data Race:** Unsynchronized access — **Undefined Behavior** in C/C++.
- **Logical Race:** Ordering error (e.g., a dependency violation — a thread consumes a result before another thread has finished producing it).

13.2 The Trade-off: Correctness vs. Scalability

- **Correctness:** Synchronization enforces order and prevents races.
- **Overhead:** Atomic instructions flush store buffers (latency).
- **Contention:** Threads wait for locks (serialization).
- **Amdahl's Law:** Synchronization increases the serial portion of code, placing a hard ceiling on achievable speedup.

> ⚡ **Key Insight**
>
> Synchronization is necessary for correctness, but each mechanism has a different cost. The golden rule: use the **lightest correct mechanism** available. Prefer reduction over atomic over critical, in that order of performance.

14. Synchronization Constructs in Detail

14.1 critical — Ensures a block executes by only one thread at a time

Incorrect — Read-Modify-Write Race on count

```
int count = 0;
#pragma omp parallel for
for (int i = 0; i < 1000; i++) {
    // Read-Modify-Write Race — count++ is NOT atomic
    count++;
}
// Result: < 1000 (Random, non-deterministic)
```

Correct — Named critical section serializes the update

```
int count = 0;
#pragma omp parallel for
for (int i = 0; i < 1000; i++) {
    #pragma omp critical(my_lock)
    {
        count++;
    }
}
// Result: always 1000
```

Performance notes:

- **Named:** `critical(name)` allows distinct sections to run concurrently with each other.
- **Bottleneck:** This loop becomes effectively serial. **Avoid large critical sections.**

14.2 atomic — Uses hardware primitives for lightweight synchronization

Uses hardware primitives (Compare-And-Swap, Load-Linked/Store-Conditional) for lightweight synchronization on single memory locations.

Syntax: `#pragma omp atomic [update|read|write|capture]`

Atomic counter — maps to a single hardware instruction (e.g., LOCK XADD on x86)

```
int count = 0;
#pragma omp parallel for
for (int i = 0; i < 100000; i++) {
    // Hardware instruction — no OS lock involved
    #pragma omp atomic
    count++;
}
```

Property	atomic	critical
Overhead	Low — no OS lock	High — OS lock mechanism
Scope	Single scalar variable only	Can protect arbitrary logic blocks
Supported ops	+=, -=, *=, ++, --, etc.	Any code block
Performance note	Much faster than critical	Can protect complex logic
Contention	Cache line bouncing at high contention	Thread queuing/serialization

Performance note: much **faster** than critical, but high contention on a single atomic variable still degrades performance (cache line bouncing).

14.3 barrier — A synchronization point where all threads must arrive before any can proceed

Implicit Barriers: Occur automatically at the end of `parallel`, `for`, `single`, `sections` constructs.

Explicit barrier separating two dependent phases

```
#pragma omp parallel
{
    setup_data(omp_get_thread_num()); // Phase 1

    #pragma omp barrier               // Explicit sync — wait for ALL threads

    process_neighbor_data(omp_get_thread_num()); // Phase 2
}
```

Cost: The **fastest thread waits for the slowest thread** (Load Imbalance). If Phase 2 reads data written by other threads in Phase 1, the barrier is **mandatory** for correctness.

14.4 nowait — Removes the implicit barrier at end of a work-sharing construct

nowait clause example

```
#pragma omp parallel
{
    // Loop A
    #pragma omp for nowait
    for (int i = 0; i < N; i++) A[i] = calc(i);

    // Loop B (Starts immediately without waiting for all threads to finish
Loop A)
    #pragma omp for
```

```
    for (int j = 0; j < M; j++) B[j] = A[j] + 1;
}
```

Analysis:

- **Improvement:** Reduces idle time if threads finish Loop A early.
- **Risk: Data race** if Loop B depends on Loop A's results (Read-After-Write). In this example, Loop B reads `A[j]` written by Loop A — this is actually **unsafe** as written.

14.5 single vs master — revisited

single with nowait and master example

```
#pragma omp parallel
{
    #pragma omp single nowait
    printf("Init done by one thread, others don't wait.\n");

    #pragma omp master
    printf("I am thread 0. No barrier here by default.\n");
}
```

15. Race Conditions in Depth and False Sharing

15.1 Example: The Reduction Mistake

A common error is manually accumulating a sum without locks or reduction clauses:

UNDEFINED BEHAVIOR — data race on sum

```
// UNDEFINED BEHAVIOR
int sum = 0;
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    sum += array[i]; // Data Race!
}
// Output is non-deterministic (changes every run)
```

Fix: `#pragma omp parallel for reduction(+:sum)` — Reduction creates private copies for each thread and merges them at the end.

15.2 False Sharing: The Silent Performance Killer

The Hardware Reality:

- CPUs read/write memory in **cache lines** (typically 64 bytes on modern x86 processors).
- **Coherence (MESI protocol):** If Core A writes to a cache line, Core B's copy is **invalidated** and must be re-fetched from a higher (slower) level of the memory hierarchy.
- **Scenario:** Independent variables residing on the **same cache line** — threads are not competing over the same data, but over the same cache line (hence 'false' sharing).

Slow — False Sharing: 4 independent variables on ONE 64-byte cache line

```
int counts[4]; // 4 ints = 16 bytes total
// Fits entirely in 1 cache line!

#pragma omp parallel num_threads(4)
{
    int id = omp_get_thread_num();
    for (int i = 0; i < N; i++)
        counts[id]++;
```

```
    // Every write by any core invalidates the SAME cache line in all other
cores
    // Cores fight for the cache line — 10x+ slowdown possible
}
```

Fast — Padding: each variable on its own dedicated cache line

```
struct Padded {
    int val;
    char pad[60]; // 4 bytes (int) + 60 bytes padding = 64 bytes = one full
cache line
};
struct Padded counts[4];

#pragma omp parallel num_threads(4)
{
    int id = omp_get_thread_num();
    counts[id].val++;
    // Each thread's data is on its OWN cache line — no cross-core invalidation
}
```

> ⚡ **Key Insight**
>
> False sharing is a hardware-level phenomenon caused by cache line granularity. It can cause **10x or more slowdown** in tight loops and produces **no incorrect output** — the program gives the right answer, just very slowly. Diagnose with hardware performance counters (e.g., 'perf stat') and fix with padding or data restructuring.

16. SIMD: Single Instruction Multiple Data

16.1 Thread-Level (TLP) vs Data-Level Parallelism (DLP)

- **TLP (OpenMP Threads):** Multiple cores running independent instruction streams.
- **DLP (SIMD):** One core executing **one instruction on wide registers** (128/256/512 bits).

AVX-512 can process **16 floats in a single clock cycle**. Vectorization is critical for maximum performance on modern CPUs. TLP and DLP are orthogonal — combining them gives the product of both speedups.

ISA Extension	Register Width	Floats per instruction (32-bit)
SSE	128 bits	4 floats
AVX / AVX2	256 bits	8 floats
AVX-512	512 bits	16 floats

16.2 Construct: simd

Syntax: `#pragma omp simd [clauses]`. Hints the compiler to ignore vector dependencies and generate vector instructions (SSE/AVX).

Vector addition — compiled to ADDPS/VADDPS (SSE/AVX)

```
void add_vectors(float *a, float *b, float *c, int N) {
    #pragma omp simd
    for (int i = 0; i < N; i++) {
        a[i] = b[i] + c[i];
    }
}
```

SIMD Reduction — dot product

```
double dot = 0.0;
#pragma omp simd reduction(+: dot)
for (int i = 0; i < N; i++) dot += a[i] * b[i];
```

16.3 Combining Threads and SIMD: #pragma omp parallel for simd

- Splits the loop among threads (**TLP**).
- Vectorizes the chunk assigned to each thread (**DLP**).

Maximum throughput: TLP across cores + DLP within each core

```
#pragma omp parallel for simd aligned(a, b, c: 64)
for (int i = 0; i < N; i++) {
    a[i] = b[i] * scalar + c[i];
}
```

The **`aligned` clause**:

- Tells the compiler that pointers are aligned to 64-byte boundaries.
- Allows use of **aligned load/store instructions** (VMOVAPS) which are faster than unaligned (VMOVUPS).
- **Critical requirement:** Memory must actually be allocated with alignment using `posix_memalign` or `_mm_malloc`. Applying the `aligned` clause to actually unaligned memory causes a crash (bus error / segfault).

Properly aligned memory allocation (required for aligned clause to be safe)

```
float *a;
posix_memalign((void**)&a, 64, N * sizeof(float)); // 64-byte aligned
```

17. Task-Based Parallelism

17.1 Why Tasks?

- **`for` loops require known iteration counts** — not always available.
- Tasks handle **irregular problems:** Recursion, Linked Lists, While loops.

Concept: Work Stealing — A pool of tasks is generated; idle threads "steal" tasks from the queues of busy threads.

17.2 Construct: task and taskwait

Syntax: `#pragma omp task [clauses]`. Creates a deferred unit of work that may be executed by any thread in the team, potentially asynchronously.

Recursive Fibonacci using tasks — sub-problems run concurrently

```
int fib(int n) {
    if (n < 2) return n;
    int x, y;

    // Create async task
    #pragma omp task shared(x)
    x = fib(n - 1);

    // Create async task
    #pragma omp task shared(y)
    y = fib(n - 2);

    // Wait for direct children only
    #pragma omp taskwait
    return x + y;
```

```
}

// Note: initial call must be inside a parallel region (usually single)
#pragma omp parallel
#pragma omp single
{
    result = fib(30); // One thread generates tasks; all threads execute them
}
```

When `fib(n)` is called, it spawns two tasks and suspends at `taskwait` until both complete. Those tasks may themselves spawn further tasks, building a tree of concurrent computation. `x` and `y` are declared `shared` so tasks write to the same variables the parent reads after `taskwait`.

17.3 Construct: taskloop

Purpose: Parallelize loops using **tasks instead of threads** directly. Good when loop iterations vary significantly in workload.

taskloop with grainsize — batches iterations to amortize task creation overhead

```
#pragma omp parallel
#pragma omp single
{
    // Breaks loop into tasks
    #pragma omp taskloop grainsize(50)
    for (int i = 0; i < N; i++) {
        process_irregular_data(i);
    }
}
```

Granularity: Creating a task has overhead. The `grainsize` clause groups iterations to amortize this cost. If grainsize = 1, the overhead of creating N tasks may far exceed the actual computation. The `grainsize` clause batches iterations, just as `chunksize` does for `schedule(dynamic)`.

17.4 Task Dependencies: depend clause

Concept: Build a **Directed Acyclic Graph (DAG)** of execution. The runtime enforces order based on data addresses, automatically delaying dependent tasks.

Dependency types: **in**, **out**, **inout**.

Producer-consumer with depend — runtime enforces ordering automatically, no manual locks needed

```
int x;
#pragma omp parallel
#pragma omp single
{
    // Task 1: Producer
    #pragma omp task depend(out: x)
    x = compute_init();

    // Task 2: Consumer (Waits for Task 1 automatically)
    #pragma omp task depend(in: x)
    printf("Result: %d\n", x);
}
```

Dependency Type	Meaning
depend(out: var)	This task writes var; subsequent tasks reading var must wait.
depend(in: var)	This task reads var; must wait for any preceding task that writes var.
depend(inout: var)	This task reads and writes var; both read and write dependencies apply.

Allows for complex **pipeline parallelism** without manual locks or barriers.

> ⚡ **Key Insight**
> Task dependencies express WHAT must happen before WHAT, not WHEN or on WHICH thread. The runtime scheduler handles execution order automatically, allowing maximum concurrency while guaranteeing correctness.

18. Performance Mental Model

When writing or analyzing an OpenMP program, work through this checklist in order:

- **Step 1 — Correctness first.** Identify all shared variables. Protect any read-modify-write access with atomic, critical, or reduction. Check for ordering dependencies that need barrier or taskwait.
- **Step 2 — Choose the right synchronization.** Prefer reduction over atomic over critical. Use nowait wherever you can prove independence.
- **Step 3 — Check data layout.** If multiple threads write to nearby memory locations, check whether they share a cache line. If so, add padding to eliminate false sharing.
- **Step 4 — Exploit SIMD.** For compute-intensive array loops, add simd with proper memory alignment. Combine with parallel for to use both TLP and DLP simultaneously.
- **Step 5 — Handle irregular workloads with tasks.** If iteration counts are unknown, work is recursive, or per-task cost varies widely, use task/taskloop with depend.

Recommended compilation flags — always compile with optimization and native SIMD

```
gcc -fopenmp -O2 -march=native code.c -o exec
# -march=native enables AVX2 / AVX-512 if available on the current CPU
```

19. Summary Reference Table

Concept	Construct	When to Use
Mutual exclusion (arbitrary code)	critical	Complex shared data structures
Lightweight scalar update	atomic	Counters, simple accumulators
Phase synchronization	barrier	Dependent computation phases
Remove implicit synchronization	nowait	Independent consecutive constructs
Single-thread execution (any thread)	single	I/O, initialization
Single-thread execution (master only)	master	Master-worker patterns
Cache line isolation	Struct padding	Arrays of per-thread variables
Intra-core data parallelism	simd	Array loops on a single thread
Multi-core + SIMD combined	parallel for simd	Maximum throughput on array ops
Irregular / recursive parallelism	task + taskwait	Recursive algorithms, pointer structures

Concept	Construct	When to Use
Variable-cost loop parallelism	taskloop grainsize	Loops with unpredictable iteration cost
Automatic task ordering	depend(in/out/inout)	Pipeline, producer-consumer patterns

End of OpenMP Complete Study Guide — High Performance Computing Module